

Login HAW-Rechner

- Benutzername: In eurer w-Kennung **w** durch **x** ersetzen
 - Aus wyf276@haw-hamburg.de wird **xyf276**
- Passwort: **HaW!%EURE_MATRIKELNUMMER%!**
- User wab123 mit Matrikelnummer 7654321 wird zu
- Username: xab123
- Password: HaW!7654321!

Collections

Gliederung

- Array (Wiederholung)
- Generics <T>
- Iterable
- Collection
- List
- Set
- Map

Array

```
public class Demo {  
  
    record Student(String name) {}  
  
    public static void main(String[] args) {  
        int[] someNumbers = new int[] { 1, 2, 3 };  
        Student[] someStudents = new Student[10];  
  
        someStudents[0] = new Student("Sara");  
        someStudents[1] = new Student("Jeff");  
        someStudents[2] = new Student("Anna");  
  
        for (int i = 0; i < someStudents.length; i++) {  
            System.out.println(someNumbers[i]);  
            System.out.println(someStudents[i]);  
        }  
    }  
}
```

Definition „Array“ von Oxford:

1. “an impressive display or range of a particular type of thing.”
2. “an ordered series or arrangement.”
3. “an indexed set of related elements.” (computing)

Origin



Middle English (in the senses 'preparedness' and 'place in readiness'): from Old French *arei* (noun), *areer* (verb), based on Latin *ad-* 'towards' + a Germanic base meaning 'prepare'.

Array

```
public class Demo {  
  
    record Student(String name) {}  
  
    public static void main(String[] args) {  
        int[] someNumbers = new int[] { 1, 2, 3 };  
        Student[] someStudents = new Student[10];  
  
        someStudents[0] = new Student("Sara");  
        someStudents[1] = new Student("Jeff");  
        someStudents[2] = new Student("Anna");  
  
        for (int i = 0; i < someStudents.length; i++) {  
            System.out.println(someNumbers[i]);  
            System.out.println(someStudents[i]);  
        }  
    }  
}
```

Ausgabe:

```
1  
Student[name=Sara]  
2  
Student[name=Jeff]  
3  
Student[name=Anna]  
Exception in thread "main"  
    java.lang.ArrayIndexOutOfBoundsException:  
        Index 3 out of bounds for length 3  
        at Demo.main(Demo.java:15)
```

Array

Tue Folgendes nicht!!!!!!!!!!!!!!!!!!!!1

Nutze stattdessen Arrays (oder Collections).

```
public class Room {  
  
    private Seat seat1;  
    private Seat seat2;  
    private Seat seat3;  
    private Seat seat4;  
    private Seat seat5;  
    private Seat seat6;  
    private Seat seat7;  
    private Seat seat8;  
    private Seat seat9;  
    private Seat seat10;  
    // ...  
    private Seat seat60000;  
}
```



Generics <T>

Mithilfe von Generics ist es möglich, Klassen zu erstellen, die mit einem bestimmten Datentyp arbeiten. Der Bezeichner zwischen den <> kann beliebig gewählt werden.

```
class SimpleBag<T> {  
    private T item;  
    public SimpleBag(T item) {  
        this.item = item;  
    }  
    public T getItem() {  
        return this.item;  
    }  
  
    public static void main(String[] args) {  
        SimpleBag<Integer> bagWithNumber = new SimpleBag<>(42);  
        System.out.println(bagWithNumber.getItem());  
        SimpleBag<String> bagWithName = new SimpleBag<>("Moin");  
        System.out.println(bagWithName.getItem());  
    }  
}
```

Ausgabe:

42
Moin

Java Collections Framework

(ein Ausschnitt aus [java.util](https://docs.oracle.com/javase/8/docs/api/java/util/package-summary.html))

Legende:

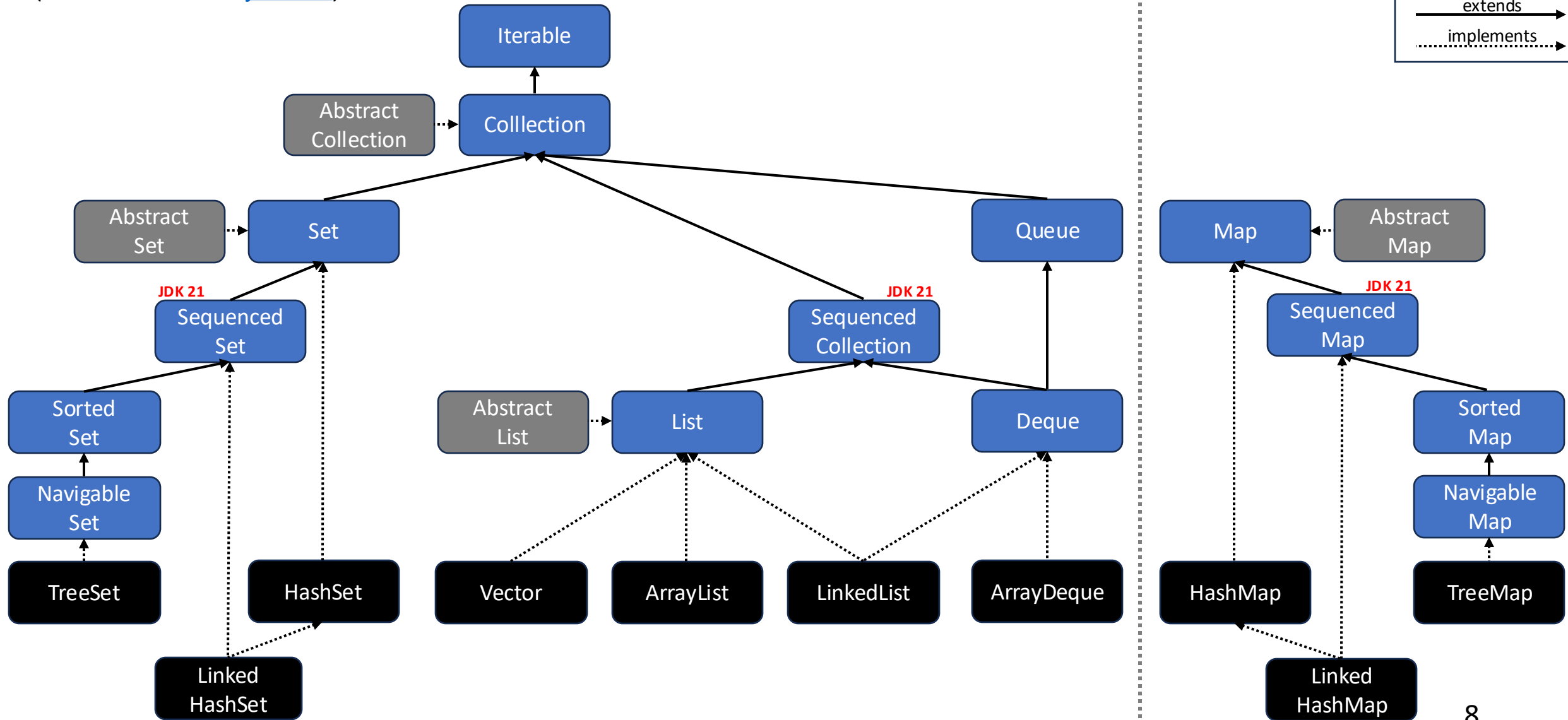
Interface

Klasse

Abstr. Klasse

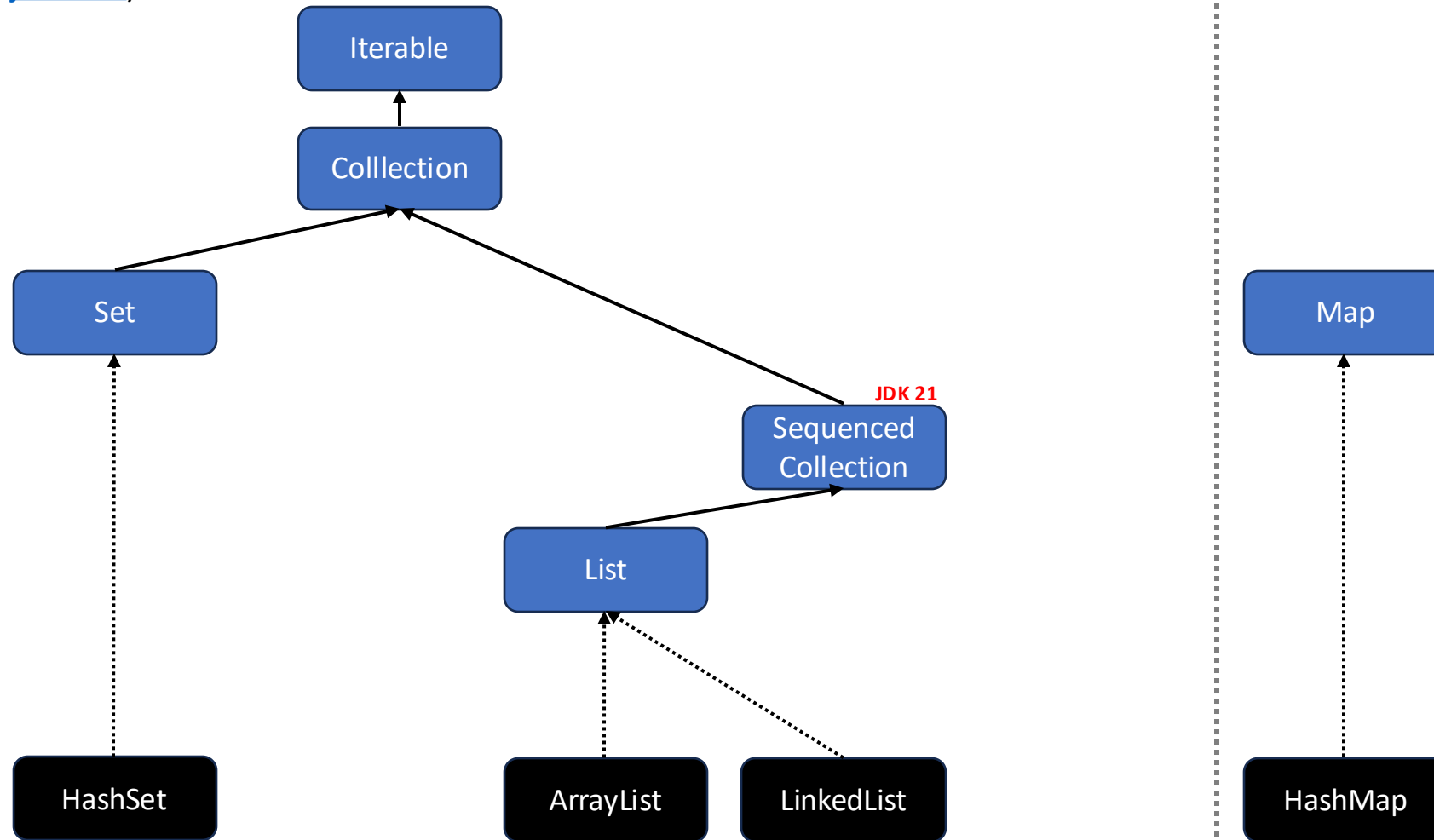
extends

implements

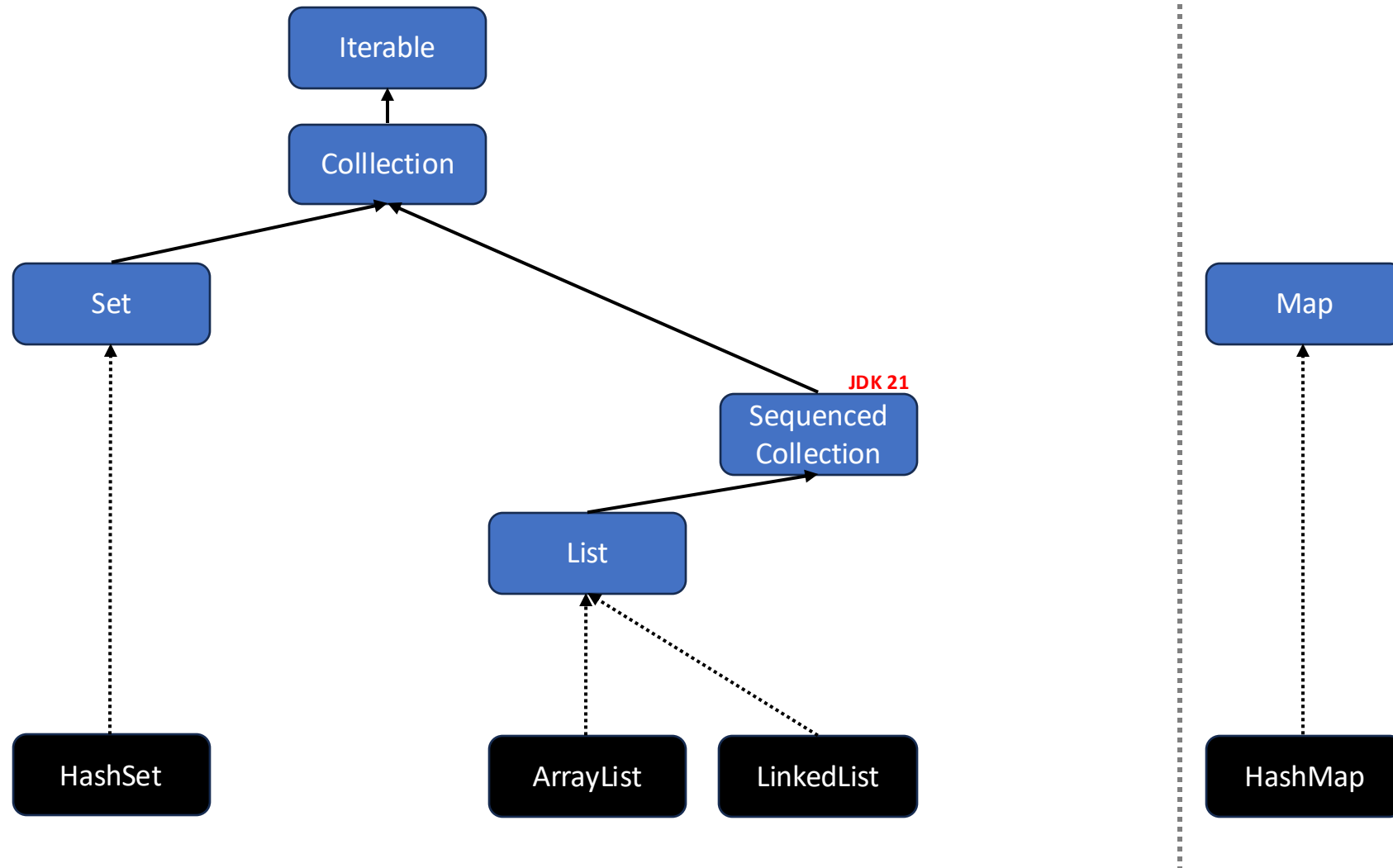


Java Collections Framework

(ein Ausschnitt aus [java.util](https://docs.oracle.com/javase/8/docs/api/java/util/))



Wir schauen uns nur einen Teil an! 😊



Iterable

Definition „iterieren“ von Oxford:

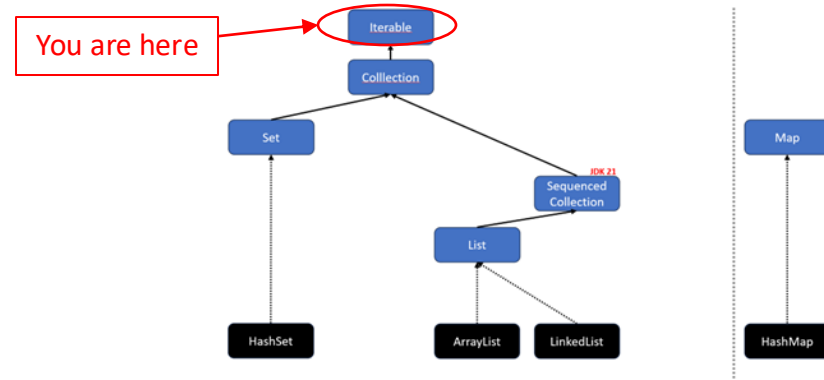
1. (ein Wort, eine Wortgruppe) im Satz wiederholen
2. wiederholen (EDV)

Wenn eine Klasse das Interface [Iterable](#) implementiert, kann man ihre Elemente mithilfe eines [Iterators](#) durchlaufen. Über die Methode `next()` erhalten wird immer das nächste Element.

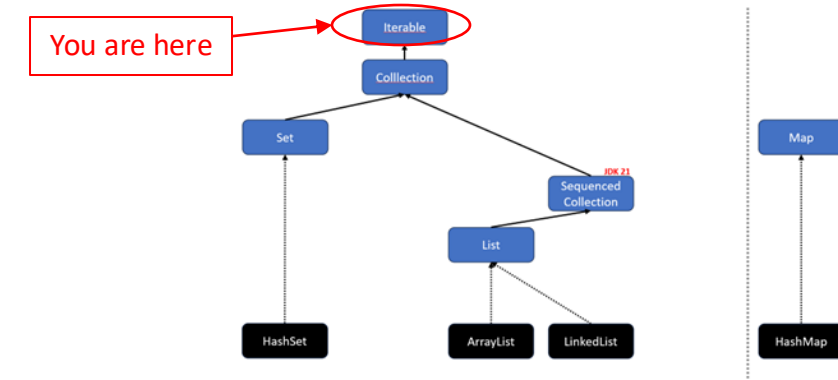
```
public class Main {  
    public static void main(String[] args) {  
        int[] numbers = {0, 1, 2, 3, 4};  
        Iterator iterator = Arrays.stream(numbers).iterator();  
        while (iterator.hasNext()) {  
            Object current = iterator.next();  
            System.out.print(current + " ");  
        }  
    }  
}
```

Ausgabe:

0 1 2 3 4



Iterable<E>



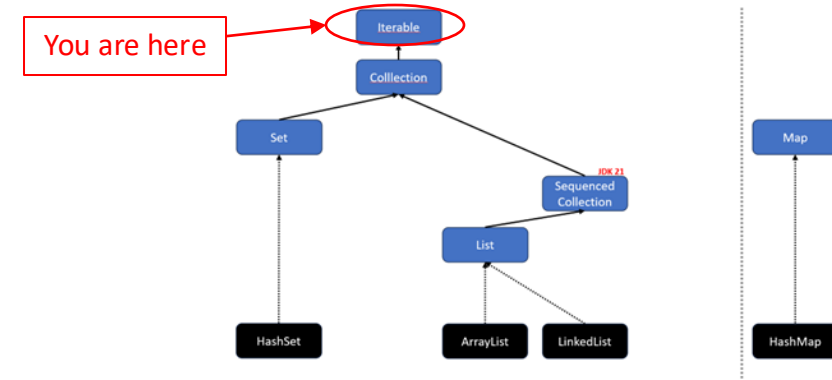
Mithilfe von Generics ist es möglich, ein `Iterable<E>` zu erstellen, das einem bestimmten Datentyp verwendet und zurückgibt.

```
public class Main {  
    public static void main(String[] args) {  
        int[] numbers = {0, 1, 2, 3, 4};  
        Iterator<Integer> iterator = Arrays.stream(numbers).iterator();  
        while (iterator.hasNext()) {  
            int number = iterator.next(); // Java unboxes Integer to int  
            System.out.print(number + " ");  
        }  
    }  
}
```

Ausgabe:

0 1 2 3 4

Iterable: for-each



Das Interface `Iterable` erlaubt es außerdem, mit dem Java-Schlüsselwort *for* über die Elemente zu iterieren. 🌀

🤪 FYI: *for* funktioniert auch mit Arrays.

```
public class Main {  
    public static void main(String[] args) {  
        int[] numbers = {0, 1, 2, 3, 4};  
        for(int number: numbers) {  
            System.out.println(number + " ");  
        }  
    }  
}
```

Ausgabe:

0 1 2 3 4

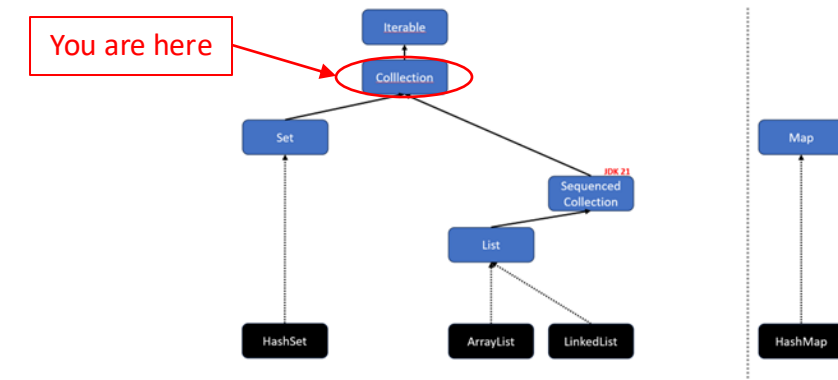
Übung: One-dimensional String-Theory

1. Erstelle ein Array "names" mit folgenden Strings:
"Chani", "Sara", "Leto"
2. Iteriere über das Array mit einem Iterator und gebe jeden Namen in GROßBUCHSTABEN in der Konsole aus.
3. Iteriere nochmal über das Array, dieses Mal mit einer for-Schleife. Ersetze hierbei in jeden Namen das "o" mit einem "i" und gebe den neuen Namen in der Konsole aus.

1. Erstelle ein Array "names" mit folgenden Strings:
"Chani", "Sara", "Leto"
2. Iteriere über das Array mit einem Iterator und gebe jeden Namen in GROßBUCHSTABEN in der Konsole aus.
3. Iteriere nochmal über das Array, dieses Mal mit einer for-Schleife. Ersetze hierbei in jeden Namen das "o" mit einem "i" und gebe den neuen Namen in der Konsole aus.

```
1 import java.util.Arrays;
2 import java.util.Iterator;
3
4 public class Demo {
5     static void main(String[] args) {
6         // 1. Erstelle ein Array "names" mit drei String-Elementen.
7         String[] names = {"Chani", "Sara", "Leto"};
8
9         // Erzeuge einen Iterator, um über das 'names'-Array zu laufen.
10        // Der Iterator ist typsicher und weiß, dass er Strings enthält (<String>).
11        Iterator<String> iterator = Arrays.stream(names).iterator();
12
13        // 2. Iteriere über das Array mit einem Iterator.
14        // Die while-Schleife läuft, solange der Iterator noch ein nächstes Element hat.
15        while (iterator.hasNext()) {
16            // Hole das nächste Element (einen Namen) aus dem Iterator.
17            String name = iterator.next();
18            // Wandle den Namen in Großbuchstaben um und gib ihn in der Konsole aus.
19            System.out.println(name.toUpperCase());
20        }
21
22        // 3. Iteriere nochmal über das Array, dieses Mal mit einer for-each-Schleife.
23        // Diese Schleife ist eine bequemere, besser lesbare Alternative zum Iterator.
24        for (String name : names) {
25            // Ersetze in jedem Namen den Buchstaben "o" durch "i" und gib das Ergebnis aus.
26            System.out.println(name.replace(target: "o", replacement: "i"));
27        }
28    }
29 }
30
```

Collection<E>



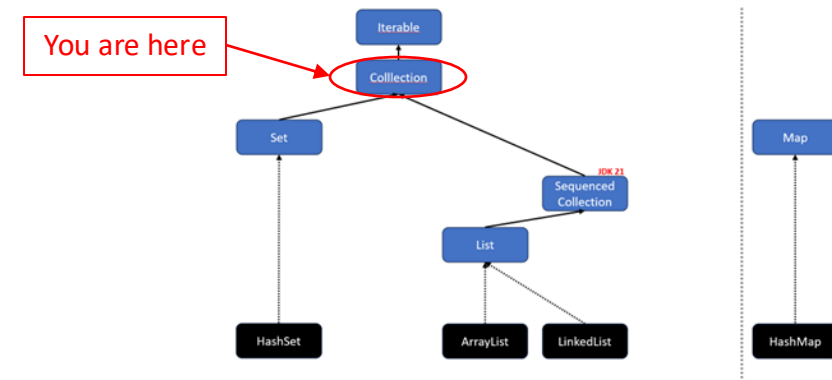
Das Interface [Collection](#)<E> bietet grundlegende Methoden, um Elemente zu hinzuzufügen oder zu entfernen.

„A collection represents a group of objects, known as its *elements*. Some collections allow duplicate elements and others do not. Some are ordered and others unordered.”

```
boolean add(E e)  
boolean addAll(Collection c)
```

```
boolean remove(Object o)  
boolean removeAll(Collection c)
```


Collection<E> – Methoden



Weitere nützliche Methoden des Collection-Interfaces:

// Enthält die Datenstruktur das Element o?

`boolean` `contains(Object o)`

// Gibt die Anzahl der enthaltenen Elemente zurück

`int` `size()`

// Gibt ein Object-Array mit allen Elementen zurück

`Object[]` `toArray()`

// Gibt einen Stream mit der Collection als Quelle zurück

`Stream<E>` `stream()` **SPOILER
ALERT!**

Collection – contains und equals

[Collection](#) bietet eine Methode *contains(Object object)*, mit der geprüft werden kann, ob die Collection (z. B. ArrayList) ein bestimmtes Objekt enthält. Für die Prüfung auf Gleichheit wird die *equals()-Methode* für jedes Objekt in der Collection aufgerufen.

Um die Gleichheitsdefinition für Objekte einer Klasse anzupassen, kann *equals()* überschrieben werden:

```
class Project {  
    private String name;  
  
    @Override // Überschreibe Default-Methode aus der Klasse Object.  
    public boolean equals(Object object) {  
        if (this == object) {  
            return true;  
        }  
        if (object == null || this.getClass() != object.getClass()) {  
            return false;  
        }  
        Project otherProject = (Project) object;  
        return Objects.equals(this.name, otherProject.name);  
    }  
}
```

Collection – contains, equals und hashCode

Wichtig: Wenn equals überschrieben wird, **muss** auch die Methode hashCode (aus der Klasse Object) überschrieben werden! Beide Methoden, equals und hashCode, müssen dieselben Attribute verwenden.

```
class Project {  
    private String name;  
  
    @Override // Überschreibe Default-Methode aus der Klasse Object.  
    public boolean equals(Object object) {  
        if (this == object) { return true; }  
  
        if (object == null || this.getClass() != object.getClass()) { return false; }  
  
        Project otherProject = (Project) object;  
        return Objects.equals(this.name, otherProject.name);  
    }  
  
    @Override // Überschreibe Default-Methode aus der Klasse Object.  
    public int hashCode() {  
        return Objects.hashCode(this.name);  
    }  
}
```



Der Hashcode wird berechnet, indem die Eigenschaften des Objekts in eine Ganzzahl umgewandelt werden.

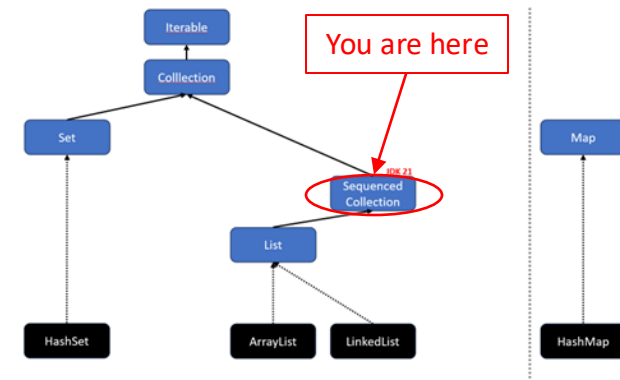
Collection – contains, equals und hashCode

Wichtig: Wenn equals überschrieben wird, **muss** auch die Methode hashCode (aus der Klasse Object) überschrieben werden! Beide Methoden, equals und hashCode, müssen dieselben Attribute verwenden.

Angenommen Project bekommt ein weiteres Attribut "private String owner;". Zwei Projekte sind gleich, wenn sowohl der Name als auch der Owner gleich sind. D. h., dass in equals und in hashCode beide Attribute verwendet werden müssen!

```
class Project {  
    private String name;  
    private String owner;  
  
    @Override // Überschreibe Default-Methode aus der Klasse Object.  
    public boolean equals(Object object) {  
        if (this == object) { return true; }  
  
        if (object == null || this.getClass() != object.getClass()) { return false; }  
  
        Project otherProject = (Project) object;  
        return Objects.equals(this.name, otherProject.name) &&  
            Objects.equals(this.owner, otherProject.owner);  
    }  
  
    @Override  
    public int hashCode() {  
        return 31 * Objects.hashCode(name) + Objects.hashCode(owner);  
    }  
}
```

SequencedCollection<E>^{JDK 21}

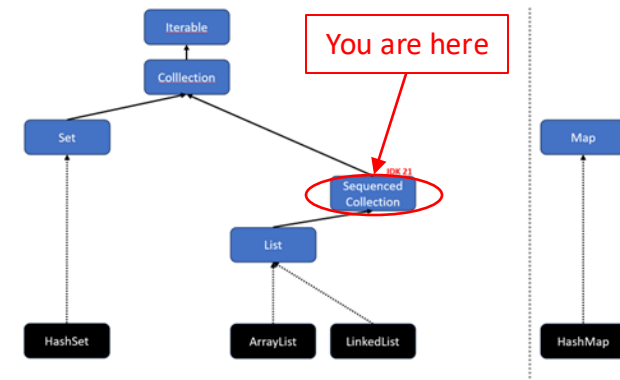


„A collection that has a well-defined encounter order, that supports operations at both ends, and that is reversible. The elements of a sequenced collection have an *encounter order*, where conceptually the elements have a linear arrangement from the first element to the last element. Given any two elements, one element is either before (closer to the first element) or after (closer to the last element) the other element.”

Das Interface [SequencedCollection](#)<E> erweitert das Interface Collection<E> und erlaubt es zudem das erste oder letzte Element zu erhalten oder zu ändern.

<code>void addFirst(E e)</code>	<code>E getFirst()</code>
<code>void addLast(E e)</code>	<code>E getLast()</code>
<code>E removeFirst()</code>	<code>SequencedCollection<E> reversed()</code>
<code>E removeLast()</code>	

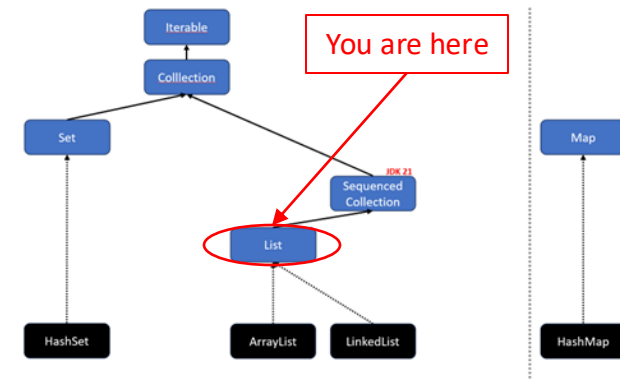
SequencedCollection<E>^{JDK 21}



“Motivation: Java’s Collections Framework lacks a collection type that represents a sequence of elements with a defined encounter order. It also lacks a uniform set of operations that apply across such collections. These gaps have been a repeated source of problems and complaints.”

	Get First Element	Get Last Element
List	<code>list.get(0)</code>	<code>list.get(list.size() - 1)</code>
Deque	<code>deque.getFirst()</code>	<code>deque.getLast()</code>
SortedSet	<code>sortedSet.first()</code>	<code>sortedSet.last()</code>
LinkedHashSet	<code>linkedHashSet.iterator().next()</code>	-

List<E>



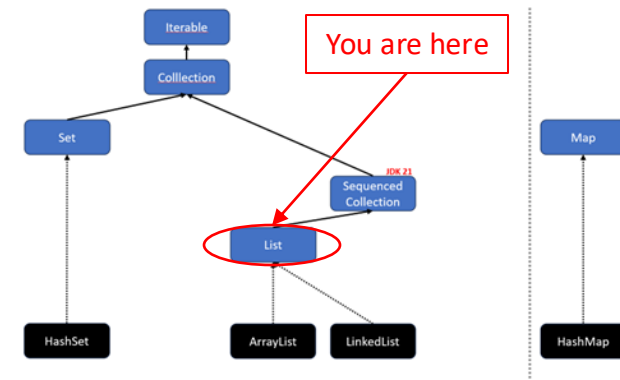
„An ordered collection (also known as a sequence). The user of this interface has precise control over where in the list each element is inserted. The user can access elements by their integer index (position in the list), and search for elements in the list. Unlike sets, lists typically allow duplicate elements.”

Das Interface [List](#)<E> erweitert das Interface `SequencedCollection<E>` und erlaubt zudem auf bestimmte Elemente direkt zuzugreifen z. B. mit `get` (siehe nächste Folie).

Häufig genutzte Implementierungen:

- `ArrayList`
- `LinkedList`

List<E> – Methoden



List<E> erweitert SequencedCollection<E> um weitere Methoden:

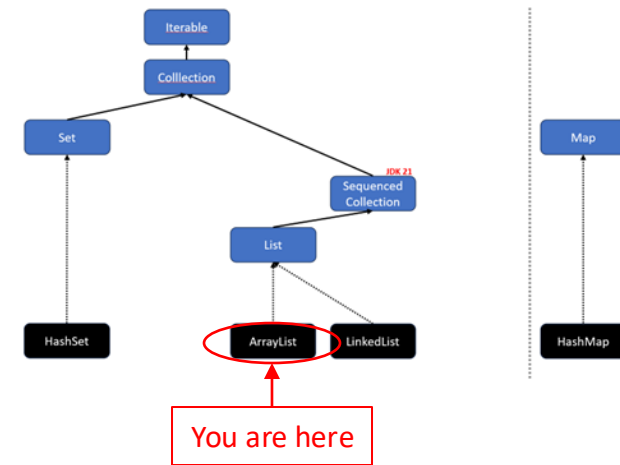
// Gibt das Element an der Position index zurück
E get(int index)

// Fügt das Element an Position index zur Liste hinzu
void add(int index, E element)

// Ersetzt das Element an der Position index und gibt das alte Element zurück
E set(int index, E element)

// Entfernt das Element an der Position index und gibt das entfernte Element zurück
E remove(int index)

ArrayList<E>



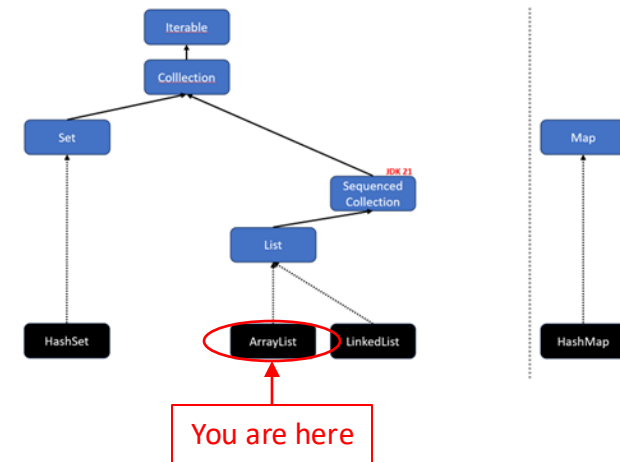
```
public class Main {  
    public static void main(String[] args) {  
        ArrayList<String> names = new ArrayList<>();  
        names.add("Sara");  
        names.add("Alia");  
        for (String name : names) {  
            System.out.println(name);  
        }  
    }  
}
```

Ausgabe:

Sara
Alia

🧐 FYI: Alternativ kannst du Folgendes schreiben: `var names = new ArrayList<String>();`
Beachte hierbei, dass der Datentyp `String` auf die rechte Seite gewandert ist,
da Java ihn sonst nicht automatisch ermittelt kann.

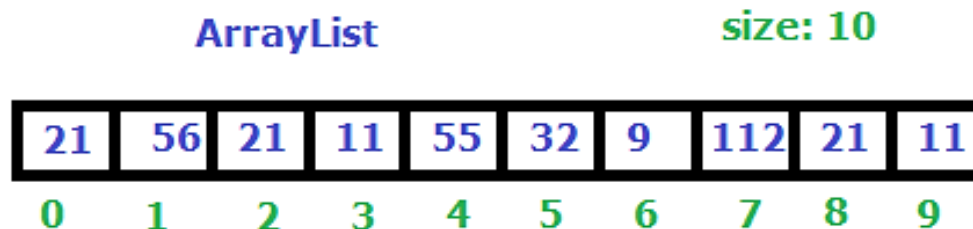
ArrayList<E>



Eine [ArrayList](#)<E> erweitert das Interface `List<E>`. Eine `ArrayList` ist im Grunde wie ein **Array mit variabler Länge** und bietet Methoden, um Elemente zu erhalten oder zu ändern (z. B. `add`, `remove`, `set`, `contains`).

```
var numbers = new ArrayList<Integer>();
```

```
numbers.addAll(List.of(21, 56, 21, 11, 55, 32, 9, 112, 21, 11));
```



Good to know:

Eine `ArrayList` verwendet intern ein gewöhnliches Array, das wenn nötig vergrößert (verdoppelt) wird. Das Vergrößern erfordert allerdings ein kostspieliges Umkopieren aller Elemente in ein größeres Array.

List.of(E...)

Alternativ gibt es auch: Set.of(E...) und Map.of(E...)

List.of(E...) ist eine praktische **Factory-Methode**, um schnell eine **Liste mit Elementen** zu erstellen.

```
public static void main(String[] args) {  
    var numbers = List.of(1, 2, 3);  
    var strings = List.of("1", "2", "3");  
    var persons = List.of(new Person("Sara"), new Person("Jeff"));  
    var personsAsObject = List.<Object>of(new Person("Sara"), new Person("Anna"));  
  
    System.out.println(numbers.getClass());  
    System.out.println(strings.getClass());  
    System.out.println(persons.getClass());  
    System.out.println(personsAsObject.getClass());  
}
```

List.of(E...)

Alternativ gibt es auch: Set.of(E...) und Map.of(E...)

Achtung: List.of(E...) gibt eine [ImmutableList](#) zurück! Methoden wie add, remove und set werfen eine UnsupportedOperationException!

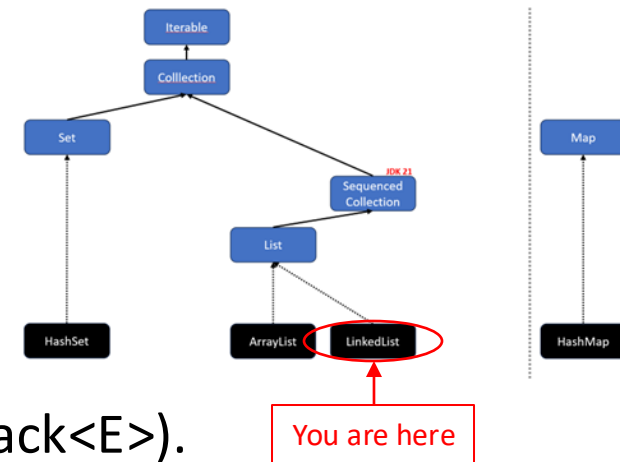
```
public static void main(String[] args) {  
    var numbers = List.of(1, 2, 3);  
    var strings = List.of("1", "2", "3");  
    var persons = List.of(new Person("Sara"), new Person("Jeff"));  
    var personsAsObject = List.<Object>of(new Person("Sara"), new Person("Anna"));  
  
    System.out.println(numbers.getClass());  
    System.out.println(strings.getClass());  
    System.out.println(persons.getClass());  
    System.out.println(personsAsObject.getClass());  
  
    numbers.add(4);  
}
```

Ausgabe:

```
class java.util.ImmutableCollections$ListN  
class java.util.ImmutableCollections$ListN  
class java.util.ImmutableCollections$List12  
class java.util.ImmutableCollections$List12  
Exception in thread "main" java.lang.UnsupportedOperationException  
    at java.base/java.util.ImmutableCollections.uoe(ImmutableCollections.java:142)
```



LinkedList<E>



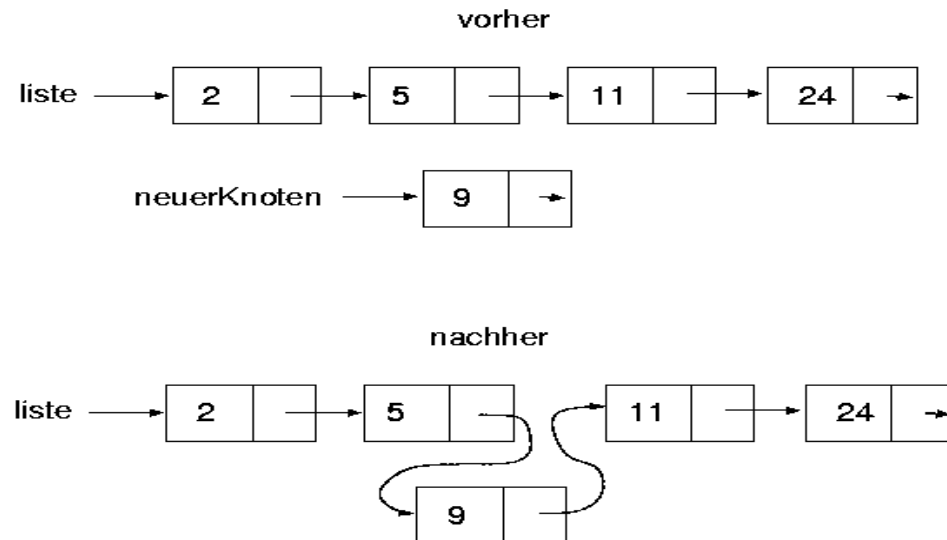
[LinkedList](#)<E> erweitert das Interface `List<E>` und `Deque<E>` (ehemals `Stack<E>`).

Eine `LinkedList` ist **optimiert für das schnelle Einfügen und Löschen von Elementen**, indem sie intern Node-Objekte verwendet, die sich referenzieren.

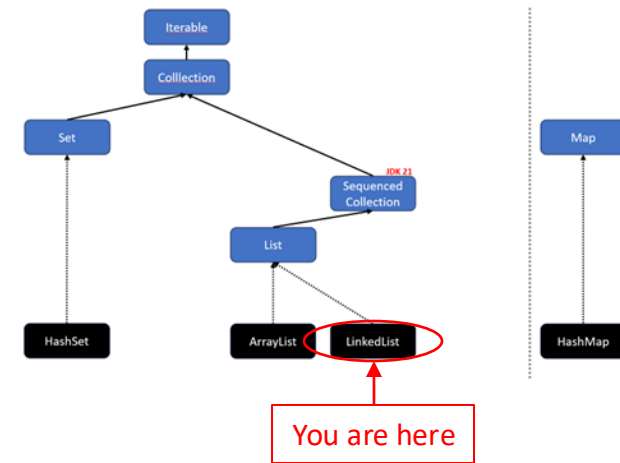
Eine Referenz lässt sich performant durch Zuweisung (`nextNode = newNode;`) „umleiten“.

So kann beim Vergrößern der `LinkedList` ein kostspieliges Umkopieren vermieden werden.

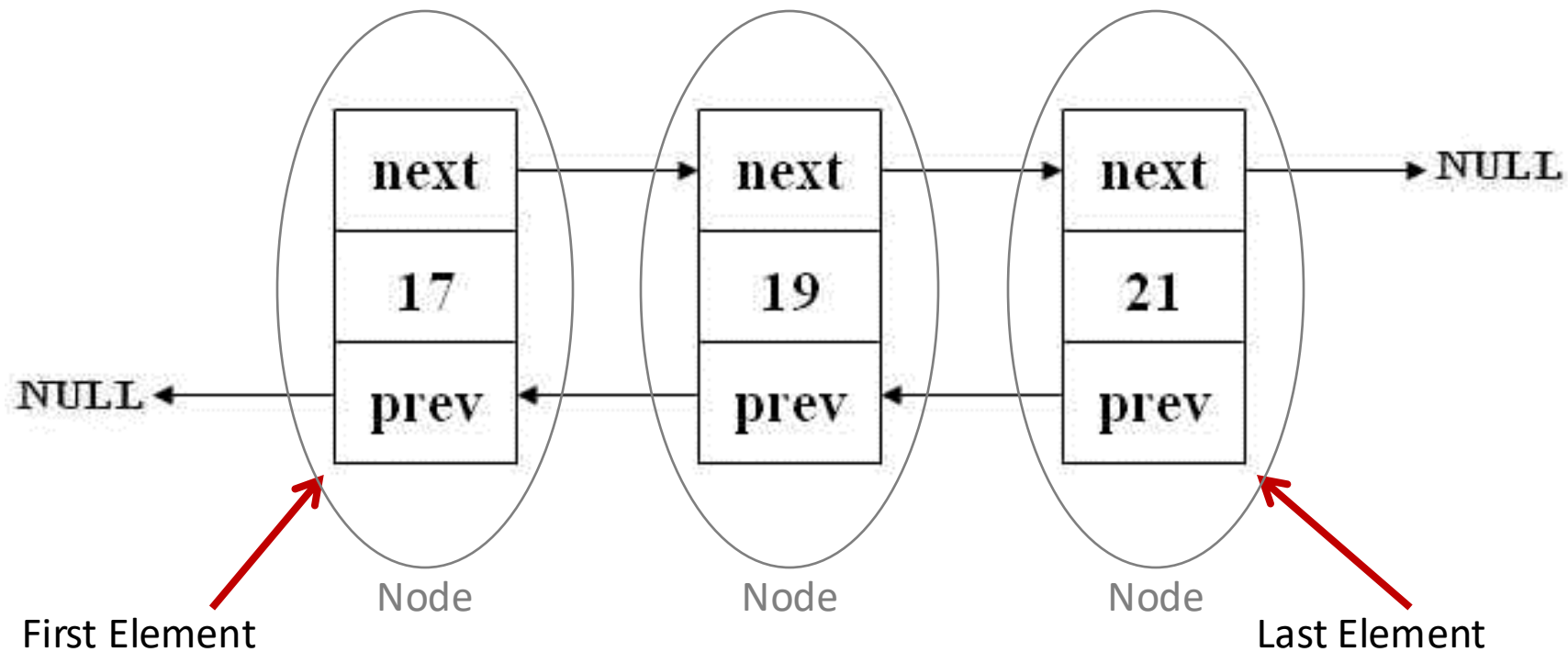
```
var numbers = new LinkedList<Integer>();
numbers.add(2);
numbers.add(5);
numbers.add(11);
numbers.add(24);
numbers.add(index: 2, element: 9);
```



LinkedList<E>



🧐 Java nutzt intern eine doppelt verkettete Liste:



Exkurs: O-Notation

- **O**: order of the function (complexity), or its growth rate.
- **n**: number of inputs, or the length of the array.

Methode	LinkedList	ArrayList
add(E element)	O(1)	O(1), worst case O(n)
add(int index, E element)	O(n), O(1) 1st and last element	O(n)
remove(int index)	O(n)	O(n)
remove(Object obj)	O(n)	O(n)
get(int index)	O(n)	O(1)
contains(Object obj)	O(n)	O(n)

🧐 Die [O-Notation](#) wird verwendet, um asymptotisches Verhalten von Funktionen und Zahlenfolgen zu beschreiben. In der theoretischen Informatik wird die Notation genutzt, um Algorithmen zu analysieren und Aussagen über Komplexität, Laufzeit und benötigten Speicherplatz in Abhängigkeit mit der Eingabegröße (Anzahl der Elemente) zu treffen. Es wird dabei immer der „Worst-Case“ betrachtet.

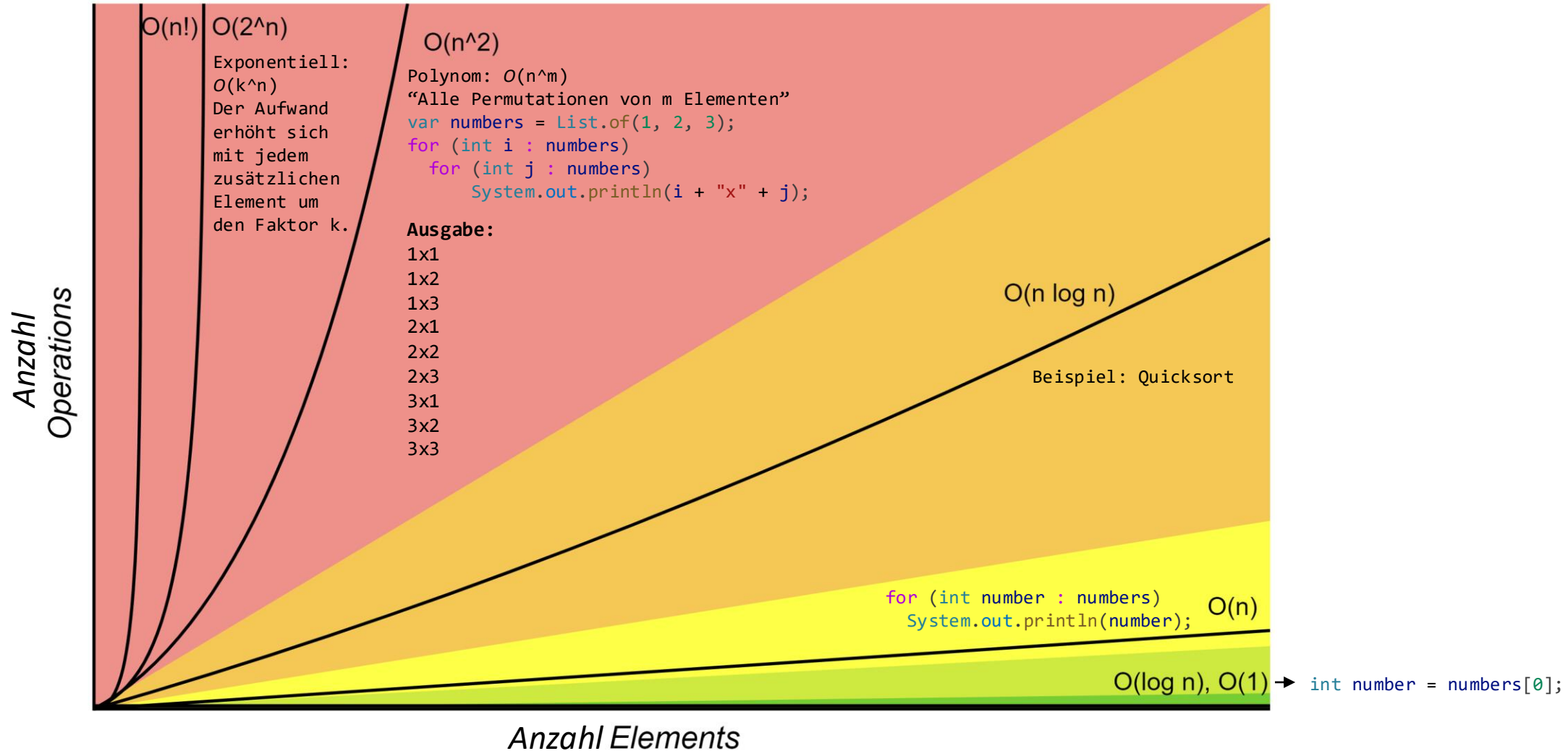
Big-O Complexity Chart

Fakultät!

Beispiel: [Traveling Salesman](#)

(kürzesten Pfad berechnen)

Horrible Bad Fair Good Excellent



Quelle: <https://www.bigocheatsheet.com/>

Übung: Kochbuch

1. Schreibe einen Record „Recipe“, der die folgenden Attribute enthält:
 - name
 - description
2. Schreibe eine main-Methode.
3. Erzeuge in der main-Methode eine Liste „recipes“ und füge einige ausgedachte Recipe-Objekte hinzu.
4. Gebe in der main-Methode alle enthaltenen Recipes in der Konsole aus.
5. Optional:
Erzeuge und überschreibe die toString()-Methode in der Klasse Recipe, um die Ausgabe zu manipulieren.

```

3 // 1. Erstelle einen Record „Recipe“ mit den Attributen 'name' und 'description'.
4 // Ein 'record' ist eine moderne, kompakte Form einer Klasse in Java.
5 // Er erstellt automatisch einen Konstruktor, die `toString()`-Methode und weitere
6 // nützliche Methoden basierend auf den deklarierten Attributen.
7 record Recipe(String name, String description) { 5 usages
8     // Innerhalb des Records überschreiben wir die standardmäßige toString()-Methode.
9     @Override
10    public String toString() {
11        // Wir geben unser eigenes, benutzerdefiniertes Format zurück.
12        return String.format("UNSER REZEPT - %s: %s", this.name, this.description);
13    }
14 }
15 public class Cookbook {
16     // 2. Schreibe eine main-Methode als Startpunkt des Programms.
17     static void main(String[] args) {
18
19         // 3. Erzeuge eine Liste „recipes“.
20         // Wir deklarieren die Variable als `List`, verwenden aber `ArrayList` als konkrete Implementierung.
21         List<Recipe> recipes = new ArrayList<>();
22
23         // Füge einige ausgedachte Recipe-Objekte zur Liste hinzu.
24         recipes.add(new Recipe(name: "Spaghetti Carbonara", description: "Ein klassisches italienisches Nudelgericht.));
25         recipes.add(new Recipe(name: "Linsensuppe", description: "Eine herzhaft und nahrhafte Suppe für kalte Tage.));
26         recipes.add(new Recipe(name: "Pfannkuchen", description: "Einfach und lecker zum Frühstück oder als Dessert.));
27
28         // 4. Gebe alle enthaltenen Recipes in der Konsole aus.
29         // Die for-each-Schleife ist der einfachste Weg, um über alle Elemente einer Liste zu iterieren.
30         System.out.println("--- Mein Kochbuch ---");
31         for (Recipe recipe : recipes) {
32             // Beim Drucken eines Objekts wird automatisch seine `toString()`-Methode aufgerufen.
33             // Da wir einen Record verwenden, ist die Ausgabe bereits gut formatiert.
34             System.out.println(recipe);
35         }
36     }
37 }

```

Bevorzuge Interfaces

Verwende für Variablen und insbesondere für Parameter Interfaces sofern vorhanden.
Wenn mehrere Interfaces zur Auswahl stehen, verwende das „höchst“-mögliche Interface.

Bei Rückgabewerten empfiehlt sich auch die Nutzung von Interfaces. Dort kommt es aber drauf an, was man anschließend mit dem Rückgabewert machen will bzw. was es dann können soll.

```
public class Demo {  
    public static void main(String[] args) {  
        Collection<Integer> numbers = new ArrayList<>(); // var numbers = ... ist prinzipiell auch okay.  
        numbers.add(1); // add-Methode aus dem Interface Collection<E>.  
        numbers.add(2);  
        numbers.add(3);  
        System.out.println(square(numbers));  
    }  
  
    public static Iterable<Integer> square(Collection<Integer> numbers) {  
        var result = new ArrayList<Integer>(numbers.size()); // Iterable<E> für den Parameter nicht  
        for (int number : numbers) { // möglich, da size() verwendet wird.  
            int square = number * number; // Je „höher“ das Interface, desto  
            result.add(square); // flexibler kann die Methode verwendet  
        } // werden.  
        return result;  
    }  
}
```

Interfaces: When Java goes wild...

Austauschbare Interfaces? Pfffff!

Was passiert, wenn wir hier Collection durch List ersetzen? 🤔

```
public static void main(String[] args) {  
    Collection<Integer> numbers = new ArrayList<>(List.of(1, 2, 3));  
    System.out.println(numbers);  
  
    numbers.remove(1); // Entfernt die Zahl 1 aus der Collection  
    System.out.println(numbers);  
}
```

Ausgabe:

```
[1, 2, 3]  
[2, 3]
```

Das Ergebnis von List.of(...) wird dem Konstruktor von ArrayList übergeben. Der Konstruktor fügt alle Elemente in die ArrayList hinzu. Warum ist das Umkopieren in dem Beispiel notwendig?

Interfaces: When Java goes wild...

Wenn wir das Interface Collection zu List ändern, erhalten wir eine unerwartete Überraschung!

```
public static void main(String[] args) {  
    List<Integer> numbers = new ArrayList<>(List.of(1, 2, 3));  
    System.out.println(numbers);  
  
    numbers.remove(1); // Entfernt die Zahl 1 aus der Collection????????????  
    System.out.println(numbers);  
}
```

Ausgabe:

```
[1, 2, 3]  
[1, 3]
```



Interfaces: When Java goes wild...

Polymorphie meets Method-Overloading.

Collection<E> Interface:

`boolean remove(Object o);`

`Collection<Integer> numbers = ...
numbers.remove(1);`

List<E> Interface:

`boolean remove(Object o);`

`E remove(int index);`

`List<Integer> numbers = ...
numbers.remove(1);`

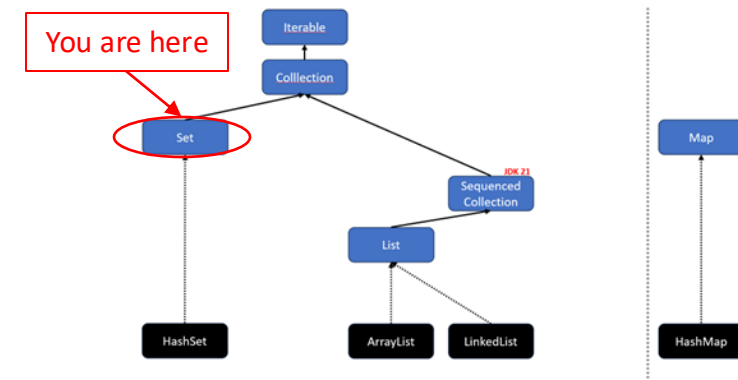


Erklärung: Wenn zwei Methoden denselben Namen und dieselbe Anzahl Parameter haben, verwendet Java die Methode, deren Datentypen direkt übereinstimmen (aka Method-Overloading).

Bei `E remove(int index);` haben die Java-Designer einen Fehler im Interface von List<E> gemacht.

Die Methode zum Entfernen von einem Element an einer bestimmten Stelle (Index) hätte nicht auch remove heißen dürfen sondern z. B. `E removeAt(int index);`. #nobodyisperfect

Set<E>



Ein [Set](#)<E> erweitert das Interface `Collection<E>` (es ist also keine `List<E>`).

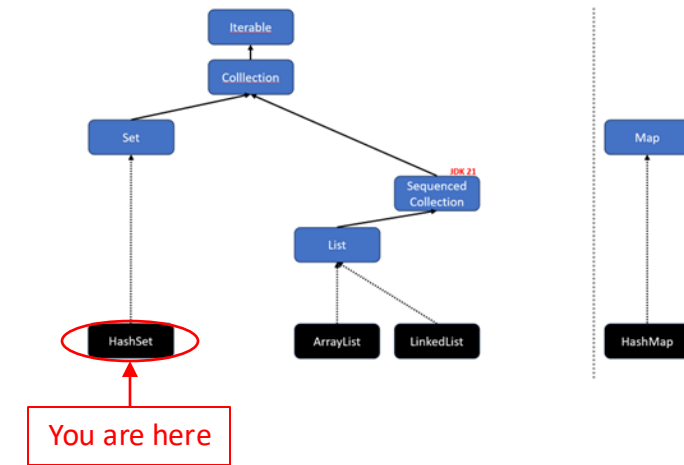
Ein Set entfernt automatisch alle Duplikate und hat keine Ordnung (manchmal).

„A collection that contains no duplicate elements. [...] As implied by its name, this interface models the mathematical *set* abstraction.”

Im Allgemeinen besitzen Sets keine Ordnung. Es gibt allerdings spezielle Implementierungen, die eine Ordnung besitzen:

- `HashSet<E>` Keine Ordnung.
- `LinkedHashSet<E>` Ordnung gemäß der Einfügereihenfolge.
- `TreeSet<E>` Natürliche Ordnung oder eigene Ordnung per [Comparator](#)<T>.

HashSet<E>



Im Gegensatz zum List-Interface besitzt das Set-Interface keine *get()*-Methode. An die Elemente gelangt man nur mittels Iterator oder for-each.

```
public class Main {  
    public static void main(String[] args) {  
        Set<Integer> uniqueNumbers = new HashSet<>();  
        uniqueNumbers.add(1); // returns true  
        uniqueNumbers.add(2); // returns true  
        uniqueNumbers.add(2); // returns false  
        uniqueNumbers.add(3); // returns true  
        for (int number : uniqueNumbers)  
            System.out.println(number);  
    }  
}
```

Ausgabe:

1
2
3

Übung: UniqueCharactersCounter

Schreibe eine Methode `countUniqueCharacters`, die die Anzahl der einzigartigen Zeichen in einem gegebenen String zurückgibt.

Verwende ein `HashSet`.

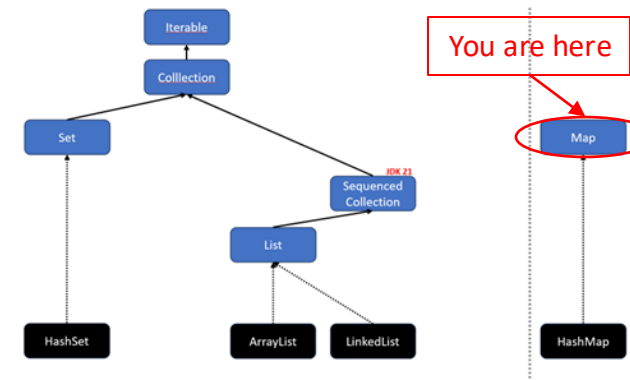
```
public static int countUniqueCharacters(String str)
```

```

1  @ public static int countUniqueCharacters(String str) { 1 usage
2      // 1. Erstelle ein HashSet, das einzelne Zeichen (Character) speichern kann.
3      // Ein HashSet ist eine Datenstruktur, die keine Duplikate erlaubt.
4      HashSet<Character> uniqueChars = new HashSet<>();
5
6      // 2. Iteriere durch jedes Zeichen des Eingabe-Strings.
7      // str.toCharArray() wandelt den String in ein Array von Zeichen um.
8      for (char c : str.toCharArray()) {
9          // 3. Füge jedes Zeichen zum HashSet hinzu.
10         // Wenn das Zeichen bereits im Set ist, passiert einfach nichts.
11         // Ist es neu, wird es hinzugefügt.
12         uniqueChars.add(c);
13     }
14
15     // 4. Gib die Größe des HashSets zurück.
16     // Die Größe entspricht der Anzahl der einzigartigen Zeichen.
17     return uniqueChars.size();
18 }
19
20
21 ▶ void main() {
22     System.out.println(countUniqueCharacters(str: "Hello World!"));
23 }

```

Map<K,V>



Eine [Map](#)<K,V> besteht aus Key-Value-Elementen (bzw. Entry).

Ein Entry<K,V> besteht aus einem Schlüssel (Key) und einem Wert (Value).

Schlüssel dürfen nur einmal in einer Map vorkommen, Werte dagegen beliebig oft.

Man kann sich eine Map wie ein flexibleres Array vorstellen...

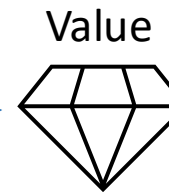
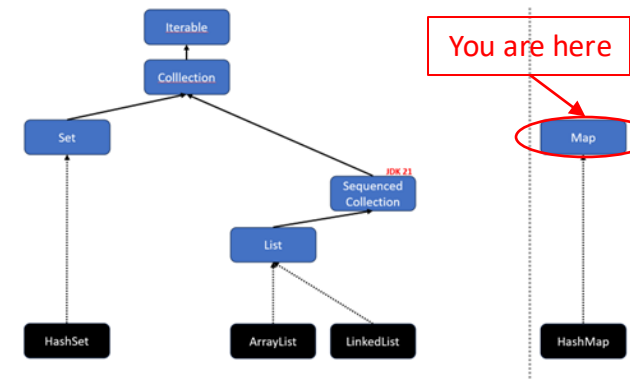
`products.get(1);` VS `products[1];`
map array

...da der Key bei einer Map etwas anderes als ein `int` sein kann (z. B. String).

 FYI: Maps erweitern in Java nicht das Interface `Iterable<E>` oder `Collection<E>`.
Maps sind trotzdem Teil des [Java Collections Frameworks](#).

Map<K,V>

Key: "Fach14"



```
class Demo {
```

```
    public static void main(String[] args) {  
        Map<String, String> cabinet = new HashMap<>();  
        cabinet.put("Fach14", "Diamond");  
        System.out.println(cabinet.get("Fach14"));  
    }  
}
```

Ausgabe:

Diamond

Map<K,V> – Methoden

// Ein Schlüssel-Wert-Paar hinzufügen
`V put(K key, V value)`

// Prüft, ob der key vorhanden ist
`boolean containsKey(Object key)`

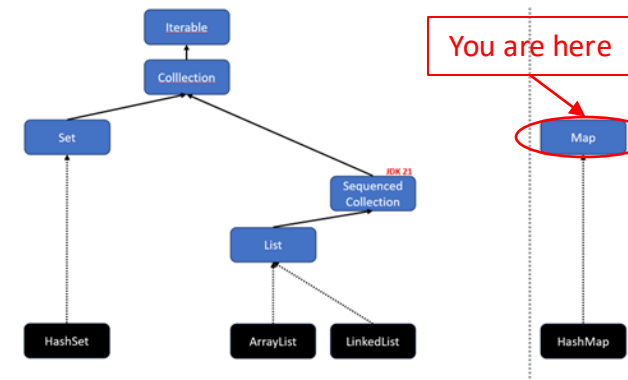
// Einen Wert über seinen Schlüssel bekommen
`V get(Object key)`

// Ein Schlüssel-Wert-Paar entfernen
`V remove(Object key)`

// Ein Set aller Schlüssel der Map erhalten
`Set<K> keySet()`

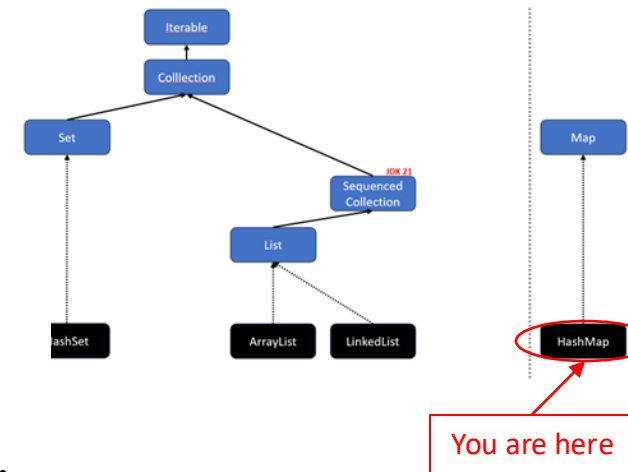
// Eine Collection aller Werte der Map erhalten
`Collection<V> values()`

// Ein Set von Paaren aus Schlüssel und Werten erhalten
`Set<Map.Entry<K, V>> entrySet()`



HashMap<K,V>

```
public class Main {  
    public static void main(String[] args) {  
        Map<String, LocalDate> projectDeadlines = new HashMap<>();  
  
        projectDeadlines.put("PR2", LocalDate.of(2023, 06, 10));  
        projectDeadlines.put("InfC", LocalDate.of(2023, 06, 12));  
  
        System.out.println(projectDeadlines.get("PR2"));  
  
        for (String projectName : projectDeadlines.keySet()) {  
            System.out.println(projectName);  
        }  
  
        for (LocalDate deadline : projectDeadlines.values()) {  
            System.out.println(deadline);  
        }  
    }  
}
```



Ausgabe:

```
2023-06-10  
InfC  
PR2  
2023-06-12  
2023-06-10
```

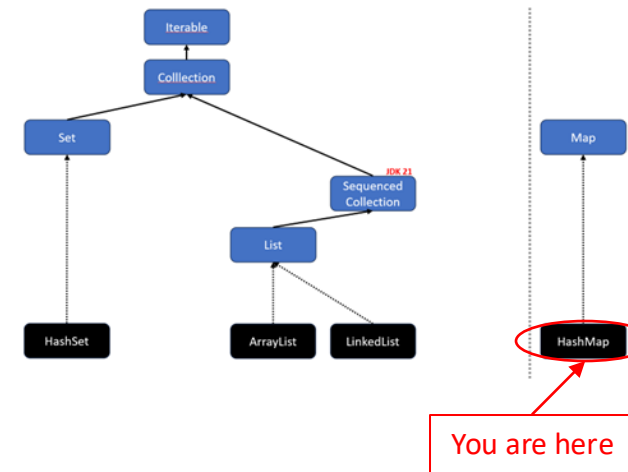
HashMap – EntrySet<K,V>

```
public class Main {  
    public static void main(String[] args) {  
        Map<String, LocalDate> projectDeadlines = new HashMap<>();  
        projectDeadlines.put("PR2", LocalDate.of(2023, 06, 10));  
        projectDeadlines.put("InfC", LocalDate.of(2023, 06, 12));  
  
        for (Entry<String, LocalDate> projectDeadline : projectDeadlines.entrySet()) {  
            System.out.println(projectDeadline.getKey() + "->" + projectDeadline.getValue());  
        }  
    }  
}
```

🧐 FYI: `Entry<String, LocalDate>` kann man hier auch mit `var` ersetzen.

Ausgabe:

```
InfC->2023-06-12  
PR2->2023-06-10
```



You are here

Java Collections Framework

(ein Ausschnitt aus [java.util](https://docs.oracle.com/javase/8/docs/api/java/util/package-summary.html))

Legende:

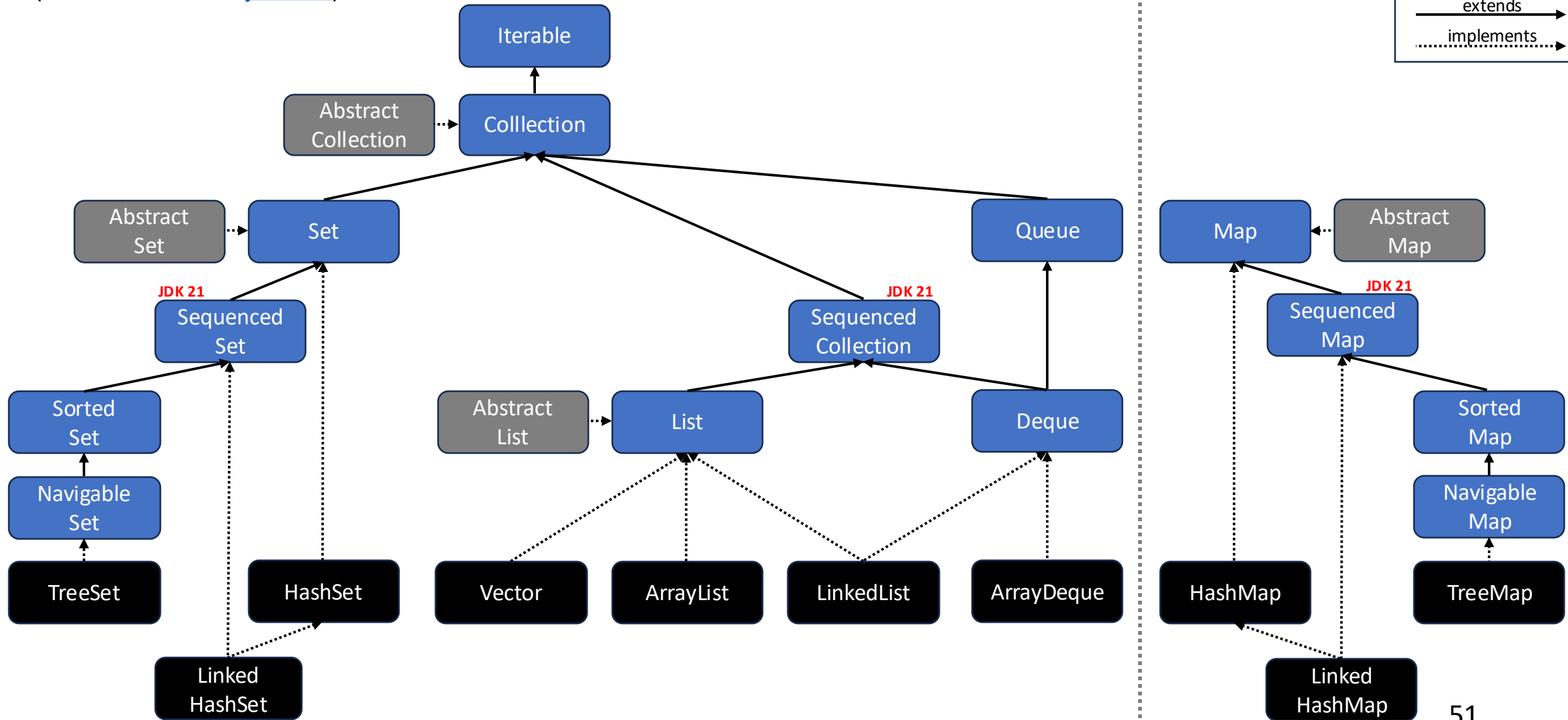
Interface

Klasse

Abstr. Klasse

extends

implements



Collections Util-Klasse

Die [Collections](#)-Klasse (beachte das s am Ende) ist eine Utility-Klasse, die hilfreiche statische Methoden für das Arbeiten mit Collections und Maps bereitstellt. Unter anderen sind folgende Methoden enthalten:

- Sortieren (sort)
- Reihenfolge umdrehen (reverse)
- Mischen (shuffle)
- Zwei Elemente tauschen (swap)
- Finden des Minimums oder Maximums (min / max)
- Unveränderliche Collections (z. B. `unmodifiableList`)
- Thread-Safe Collections (z. B. `synchronizedList`)



Collections Util-Klasse – Beispiele

```
class Demo {  
    public static void main(String[] args) {  
        List<Integer> numbers = new ArrayList<>(List.of(1, 2, 3, 4, 5));  
  
        Collections.reverse(numbers);  
        System.out.println(numbers);  
  
        Collections.swap(numbers, 1, 3);  
        System.out.println(numbers);  
  
        Integer min = Collections.min(numbers);  
        System.out.println(min);  
    }  
}
```

Ausgabe:

```
[5, 4, 3, 2, 1]  
[5, 2, 3, 4, 1]  
1
```

Beachte, dass die meisten Methoden in Collections die übergebene Collection modifiziert, es wird also **keine Kopie bzw. keine neue Collection** erzeugt.

Chaos im Collection Framework in Java

- Java ist eine recht konservative Programmiersprache. Änderungen werden nur behutsam und langsam eingeführt (z. B. `SequencedCollection`).
- Der Vorteil hierbei ist, dass Java-Programme eine hohe Abwärtskompatibilität haben, d. h., ältere Java-Programme laufen wahrscheinlich ohne Änderungen auch mit dem neuesten JDKs (sowas mögen insbesondere Unternehmen).
- Der Nachteil ist, dass moderne Sprachelemente aus anderen Programmiersprachen häufig später in Java Einzug halten.

Nächste Woche:

- **Lamda () ->**
- **Streams .stream();**

Übung am 23.10.25 startet mit einem Wiederholungsblock zu
UML